# Cursor: An Agent Harness Around Rented Models

What is explicit, what went latent, and where the seams show

**Greyson McReynolds, Michael Incorvaia, Heeba Merchant, Canting Zhu**

ECE 6256 / CS 6604 Conversational AI · Prof. Larry Heck · Case-Study Fair A, System A4 · Georgia Institute of Technology

## The System and the Question

Cursor (Anysphere) is an AI code editor forked from VS Code. It wraps frontier models from Anthropic, OpenAI, Google and xAI, plus its own models (Tab, Composer 2), in an agent harness that reads, searches, edits and runs a developer's repo. The "world" the dialogue acts on is the codebase and the terminal.

**Standing question:** which boxes of the classical decomposition (understanding, state, policy, generation, interfaces) are explicit in Cursor, which went latent inside a model, and which are missing?

**Thesis:** Cursor rents understanding and generation, engineers an explicit belief state around them, and is pulling policy in-house. The moat is the harness, not the model.

## Decomposition Map

| Box | Status | In Cursor | Owner |
|---|---|---|---|
| Understanding | Both | Trained embedding model + fast index ground the request; the semantics stay latent | Greyson |
| State | Both | Fixed static context and file-based updates; the model picks what else to fetch | Greyson |
| Policy | Split | Model picks the next tool; Cursor trains it (Tab, Composer) and fences it | Michael |
| Generation | Latent | Output lands as diffs the user reviews | Michael |
| Interfaces | Explicit, per vendor | Edit format and prompts tuned per model; typed tool errors | Michael |
| Missing | Missing | Grounding outside the repo; a real measure of task success | Heeba, Canting |

*Table 1. Each box of the classical pipeline, mapped to how Cursor implements it [1]–[5].*

## Understanding + State

### Always sent

- System prompt + tool list
- OS and git status
- Current and recent files
- Skill names, MCP tool names

### Fetched when needed

- Semantic search + grep hits
- Long tool output, as a file
- Chat history file after a summary
- Terminal sessions, as files
- Full skill files, MCP tool details

**+12.5%**
answer accuracy with semantic search, average across models (6.5–23.5%) [1]

**−46.9%**
total agent tokens on MCP runs, with tools loaded on demand [2]

## Policy: Latent, but Trained and Fenced

In L3's POMDP view the policy chooses ask, confirm or act. In Cursor the act is a tool call, chosen inside the model, so the policy is latent; Cursor shapes it at three levels.

**Tab** — Hand-written reward
Two actions, suggest or stay silent: +0.75 accept, −0.25 reject, 0 silence. L3's r(s,a), written down [4].
**28% ↑** accept rate; 21% fewer suggestions

**Composer** — RL in Cursor's harness
Trained with RL on the product's own tools in sandboxes [5]; Composer 2 learns to write its own summaries [9].
**50% ↓** compaction error at 1/5 the tokens

**Controls** — Fences on rented models
Plan mode asks and plans before acting [10]; Router, a learned classifier, picks the model per request [11].
**~60% ↓** cost at frontier quality

**The arc:** hand-written reward (L3) → task reward in a sandbox → learned satisfaction proxy (L10). None of these rewards sees the user's actual goal.

## Exhibit: One Agent Run, Every Turn Labeled

| Calls | Box | What the agent did (Grok 4.6 High Fast, Oct 26) |
|---|---|---|
| 2–3 | U | Globs the repo and ripgreps for class Library \| fees_owed \| def report |
| 4 | S | Writes a to-do plan |
| 5–10 | S·P | Reads six files in one parallel batch |
| 14–16 | G | Edits library.py (twice) and cli.py |
| 17 | G | Writes its own tests: premium grace, $10 cap, still-out loans, report line |
| 19 | I | pytest blocked: "Terminal unavailable … sandbox policy" (t = 48 s) |
| 22 | P | Reruns outside the sandbox: 10 passed (t = 50 s) |
| 23 | P | Runs the CLI on sample data to check the report (t = 64 s), then stops |

U · understanding | S · state | P · policy | G · generation | I · interface

## Interfaces: Written per Vendor

**Edit format** — Patch format for OpenAI models, string replacement for Anthropic models; the unfamiliar one costs extra reasoning tokens and more mistakes [3].

**Prompts** — Tuned per provider and per model version [3].

**Tool errors** — Typed (InvalidArguments, Timeout, ProviderError, …); anything else is treated as a harness bug [3]. The closest thing to L3's observation back to the policy.

**The seam:** switch models mid-chat and the new one reads history another model wrote, misses the prompt cache, and may call tools it lacks. Cursor's advice: one model per chat [3].

## Seams and Economics

**APR 2025 — Ungrounded support bot**
- Support bot "Sam" invented a one-device policy to explain a logout bug; users cancelled [7].
- No grounding outside the code harness.

**JUN–JUL 2025 — Renting the model**
- Pro moved from 500 requests to $20 of usage at API rates; surprise bills, refunds [6].
- The agent sets token use; the vendor sets the price.

**MAR 2026 — Owning the model**
- Composer 2's Kimi K2.5 base was not disclosed at launch; Cursor called it "a miss" [8].
- Policy moved in-house; the foundation did not.

| Model ($ per 1M tokens) | Input | Output | vs Composer 2 |
|---|---|---|---|
| Composer 2 (Standard) | $0.50 | $2.50 | 1X |
| Claude Opus 4.6 | $5.00 | $25.00 | 10X |
| GPT-5.4 | $2.50 | $15.00 | 5-6X |

*Table 2. List prices at Composer 2's launch [8, 12]. Cursor's own model costs a tenth of Opus per token.*

## Measured Behavior

**Task:** add late fees to a small Python library repo; 6 visible tests, 9 hidden tests the agent never sees. Same prompt; one run each.

| Run | Interface | Time (s) | Tool calls | Hidden tests |
|---|---|---|---|---|
| Grok 4.6 High Fast | Editor | 74 | 19 | 9 / 9 |
| Auto (model not disclosed) | CLI | 33.9 | 14 | 9 / 9 |

**Self-report vs. measured:** asked afterward, Auto estimated its run at 90 s, 2.65× the measured 33.9 s, while counting its 14 tool calls correctly.

*Table 3. Our runs, Oct 2026; one run each, different interfaces, so not a model ranking. The Free account rejected named models, and dollar costs were unavailable.*

## Conclusions

- **Explicit:** state and interfaces, engineered per vendor
- **Latent but owned:** policy, trained inside Cursor's own harness
- **Missing:** grounding outside the repo; a measure of the user's real goal
- **Measured:** Auto passed 9/9 tests but estimated 90 s vs. 33.9 s recorded
- **Limit**: one task and mixed modes cannot isolate a harness advantage

## References

[1] Cursor, "Improving agent with semantic search," Nov. 2025. cursor.com/blog/semsearch
[2] Cursor, "Dynamic context discovery," Jan. 2026. cursor.com/blog/dynamic-context-discovery
[3] Cursor, "Continually improving our agent harness," Apr. 2026. cursor.com/blog/continually-improving-agent-harness
[4] Cursor, "Improving Cursor Tab with online RL," Sep. 2025. cursor.com/blog/tab-rl
[5] Cursor, "Composer: building a fast frontier model with RL," Oct. 2025. cursor.com/blog/composer
[6] Cursor, "Clarifying our pricing," Jul. 2025. cursor.com/blog/june-2025-pricing
[7] Fortune, "Cursor's customer support AI went rogue," Apr. 2025.
[8] DataCamp, "Composer 2," Mar. 2026. datacamp.com/blog/composer-2
[9] Cursor, self-summarization research post, Mar. 2026. cursor.com/blog/self-summarization
[10] Cursor, Plan Mode post, Oct. 2025. cursor.com/blog/plan-mode
[11] Cursor, Cursor Router launch post, Jul. 2026. Cursor.com/blog/router
[12] Vantage, "Cursor's Composer 2: What it means for your AI coding costs," Mar. 2026. vantage.sh/blog/cursor-composer-2